🛒 Algo Framework — Examen Algorithmique

Objectif

Construire un **framework frontend TypeScript from scratch** (sans React, Vue, ou toute librairie externe) qui met en œuvre les concepts algorithmiques du cours.

> **Seul outil autorisé** : Vite.js comme bundler. Aucun module Node.js externe.

📋 Cahier des charges

Pages obligatoires (3 minimum)

Page	Route	Description
Accueil	`/`	Présentation de l'app + liens
Liste des produits	`/products`	Tableau avec recherche + tri
Détail produit	`/product/:id`	Fiche complète d'un produit

Patterns à implémenter

Pattern	Fichier	Cours associé
Stack Navigation (History API)	`src/router/stackRouter.ts`	Séance 2 (Pile)
Virtual DOM (hyperscript `h`)	`src/framework/vdom.ts`	Séance 3 (Récursivité)
Renderer (VNode → DOM)	`src/framework/renderer.ts`	Séance 3 (Récursivité)
Diffing / Patching	`src/algorithms/diff.ts`	Séance 3 (Récursivité)
Tri par insertion	`src/algorithms/sorting.ts`	Séance 5 (Tri)
Tri rapide	`src/algorithms/sorting.ts`	Séance 5 (Tri)
Tri fusion	`src/algorithms/sorting.ts`	Séance 5 (Tri)
Recherche linéaire	`src/algorithms/search.ts`	Séance 4 (Recherche)
Recherche binaire	`src/algorithms/search.ts`	Séance 4 (Recherche)

🚀 Démarrage

```
# Initialiser le projet (Vite + TypeScript uniquement)
npm create vite@latest . -- --template vanilla-ts
npm install

# Vérifier les types TypeScript
npx tsc --noEmit
```

```
# Lancer le serveur de développement
npx vite

# Build de production
npx vite build
```

> Vous partez de zéro : à vous de créer l'arborescence ci-dessous et d'implémenter chaque fichier.

📁 Structure de l'examen (à créer)

```
examen/
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
├── src/
│   ├── main.ts
│   ├── types/
│   │   └── index.ts
│   ├── data/
│   │   └── products.ts
│   ├── algorithms/
│   │   ├── sorting.ts
│   │   ├── search.ts
│   │   └── diff.ts
│   ├── framework/
│   │   ├── vdom.ts
│   │   └── renderer.ts
│   ├── router/
│   │   └── stackRouter.ts
│   └── pages/
│       ├── home.ts
│       ├── productList.ts
│       └── productDetail.ts
```

📝 Instructions

1. Implémenter les algorithmes

Créez les fichiers de `src/algorithms/` et implémentez les signatures suivantes :

- `sorting.ts` : `insertionSort`, `quickSort`, `mergeSort`
- `search.ts` : `linearSearch`, `binarySearch`
- `diff.ts` : `diff`, `patch` (bonus)

2. Page Liste des produits

Dans `src/pages/productList.ts`, implémentez :

- Une **barre de recherche** (linéaire OU binaire)
- Un **sélecteur d'algorithme de tri** (insertion, rapide, fusion)

- Un **tableau trié/filtré** avec temps d'exécution affiché
- Des **liens** vers la page détail

3. Navigation en pile

Dans `src/router/stackRouter.ts`, créez :

- `navigate()` — push + rendu
- `back()` — pop + rendu
- `match()` — matching des routes avec params

4. Virtual DOM (optionnel avancé)

Implémentez `h()`, `render()`, `diff()` et `patch()` pour un rendu déclaratif.

✅ Critères d'évaluation

Critère	Points
Compilation TypeScript sans erreur	/2
3 pages fonctionnelles	/3
Recherche linéaire + binaire	/2
3 algorithmes de tri fonctionnels	/3
Navigation en pile (retour fonctionnel)	/2
Diffing/Patching (bonus)	/2
Code propre, commenté, typé	/2
Total	**/16**

🎯 Bonus

- [ ] Diffing/Patching du Virtual DOM
- [ ] Temps d'exécution affiché pour chaque tri/recherche
- [ ] Tri sur colonnes cliquables
- [ ] Pagination des produits
- [ ] Filtres par catégorie
- [ ] Persistance de l'état dans l'URL (query params)
- [ ] Hooks de type React (`useState`, `useEffect`) dans le mini-framework